

Smart City Traffic Intelligence — Part 2 Report: Python Data Pipeline, Feature Engineering, Visualisation and CLI

Scope: Part 2 of the NUS SOC AI/ML/DS capstone. Input: 48,204 raw hourly I-94 records (Oct 2012 – Sep 2018). Output: a logged, reproducible pipeline producing 40,575 clean rows, a 27-feature model-ready table, four analytical figures, and a command-line application.

1. Data pipeline (`pipeline.py`)

The pipeline loads the raw CSV, validates the schema against the expected nine columns, then applies five cleaning rules. Every action is written to both the console and `pipeline.log` using the standard logging module (module-level `getLogger(__name__)`; handlers configured only in the entry point; WARNING for every row-altering action, ERROR with `exc_info=True` on failure).

Table 1 Cleaning rules and their verified effects

Rule	Action	Rows affected
Duplicate timestamps	Keep first record (traffic values never conflict within a group)	7,629 dropped (48,204 → 40,575)
temp = 0 Kelvin	Impute with the median temperature of the same month	10 imputed (Jan + Feb only)
rain_1h > 100 mm	Reset sensor spike to 0.0 (max was 9,831.3 mm)	1 reset
traffic_volume = 0	Kept — a genuine near-zero event on 2016-07-23 (whole evening 0–8 veh/h)	2 kept
holiday NaN	Restored to the raw "None" string convention	40,522 filled

Post-cleaning validation asserts unique, sorted timestamps and zero NaNs; the cleaned temperature range is –29.76 °C to +36.92 °C and maximum hourly rainfall 55.63 mm — all physically plausible.

2. Feature engineering (`feature_engineering.py`)

The clean data is transformed into a model-ready table of **40,575 rows × 27 features** (plus `date_time`, `traffic_volume` and the target):

- **Cyclical encodings** — `hour_sin/cos`, `dow_sin/cos`, `month_sin/cos`, so that hour 23 and hour 0 are neighbours rather than maximally distant integers.
- **Calendar flags** — `is_weekend` (11,596 hours), `is_rush_hour` (06–09 and 15–18; 13,399 hours).
- **Holiday fix** — the raw flag marks only the 00:00 hour of each holiday (53 rows); `is_holiday` flags all **1,203 hours** across the 53 holiday dates.
- **Weather** — one-hot encoding of `weather_main` into 11 columns; `temp_c` converted from Kelvin.
- **Target** — `congestion = 1` when `traffic_volume` ≥ Q3 (4,952): **10,152 positives (25.02%)**. The top-quartile definition gives a far better class balance for Part 3 classifiers than the stricter 5,500 threshold (15%) used in the Part 1 probability analysis.

3. Visual analysis (`visualizations.py`)

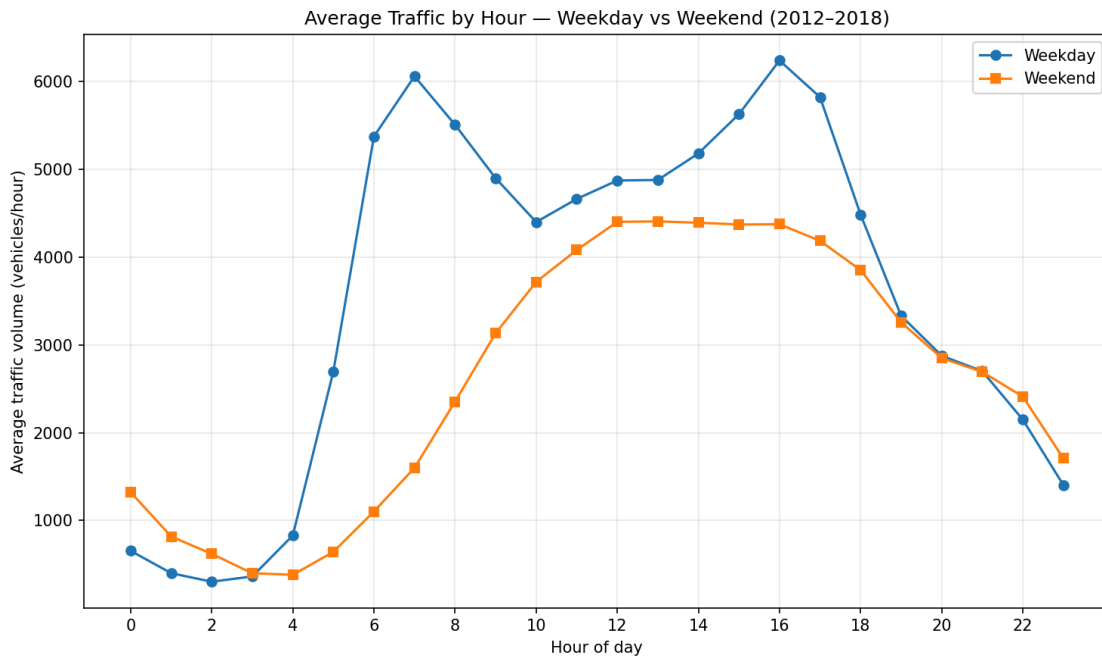


Figure 1 Weekdays show the classic twin peaks (07:00 and 16:00, $\approx 6,241$ veh/h); weekends a single midday plateau (peak 13:00, $\approx 4,408$ veh/h).

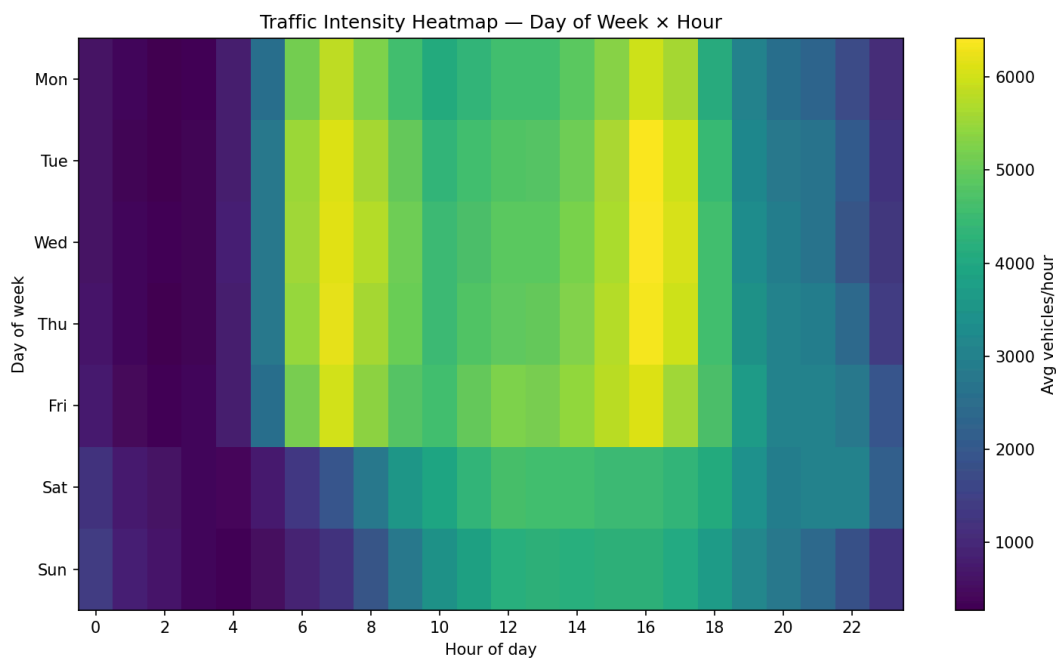


Figure 2 Day-of-week × hour intensity. The single busiest cell is Wednesday 16:00 ($\approx 6,414$ veh/h); overnight hours are near-zero every day.

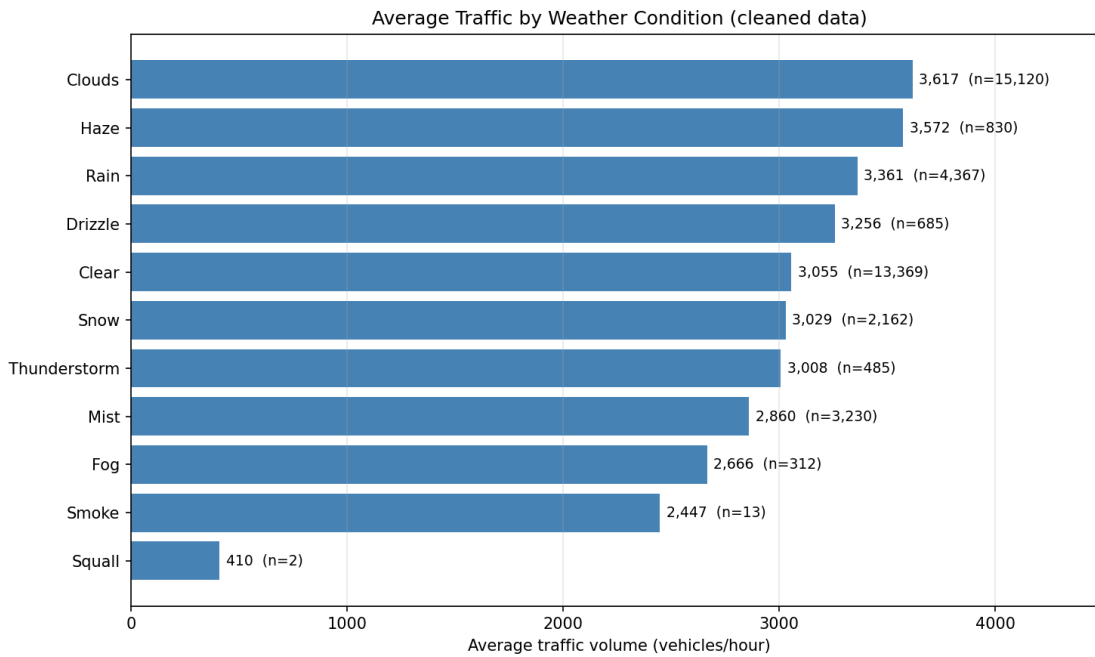


Figure 3 Average traffic by weather condition with sample sizes. Clouds highest (3,617); Fog lowest among well-sampled conditions (2,666, -19% vs overall). Squall (n=2) and Smoke (n=13) are annotated as statistically meaningless.

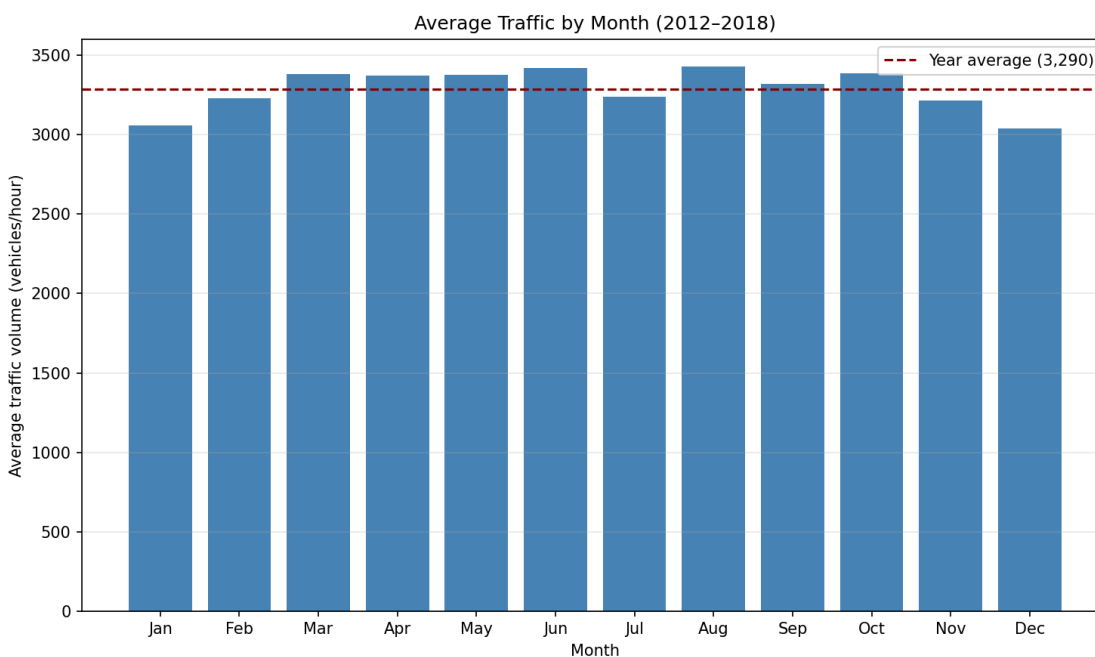


Figure 4 Monthly rhythm: a mild summer-high / winter-low pattern (August 3,430 vs December 3,040) around a year average of 3,290 veh/h.

4. Command-line application (`cli_app/main.py`)

The CLI gives the mobility team ad-hoc access to the cleaned dataset without opening a notebook. It uses `argparse` subcommands, validates input, and logs every invocation to `pipeline.log`:

```
python part2_python/cli_app/main.py summary # dataset overview + congestion stats
python part2_python/cli_app/main.py hourly # average traffic by hour (text bar chart)
```

5. Key design decisions

1. **Dedup keep="first" is safe:** within every duplicate-timestamp group the traffic values never differ — duplicates are sensor re-transmissions, not conflicting readings.
2. **Monthly-median imputation for 0 K:** preserves seasonality (Jan → 265.48 K, Feb → 265.89 K); a global median would inject summer temperatures into January.
3. **Zero-traffic rows kept:** deleting a genuine closure event would bias the model toward busier conditions.
4. **Top-quartile target:** 25% positives train classifiers far better than 15%; both thresholds are documented and traceable.
5. **Logging everywhere:** every row-altering action leaves a WARNING trail in `pipeline.log`, making the whole pipeline auditable and reproducible.